

رسالة محمد

یادگیری ماشین

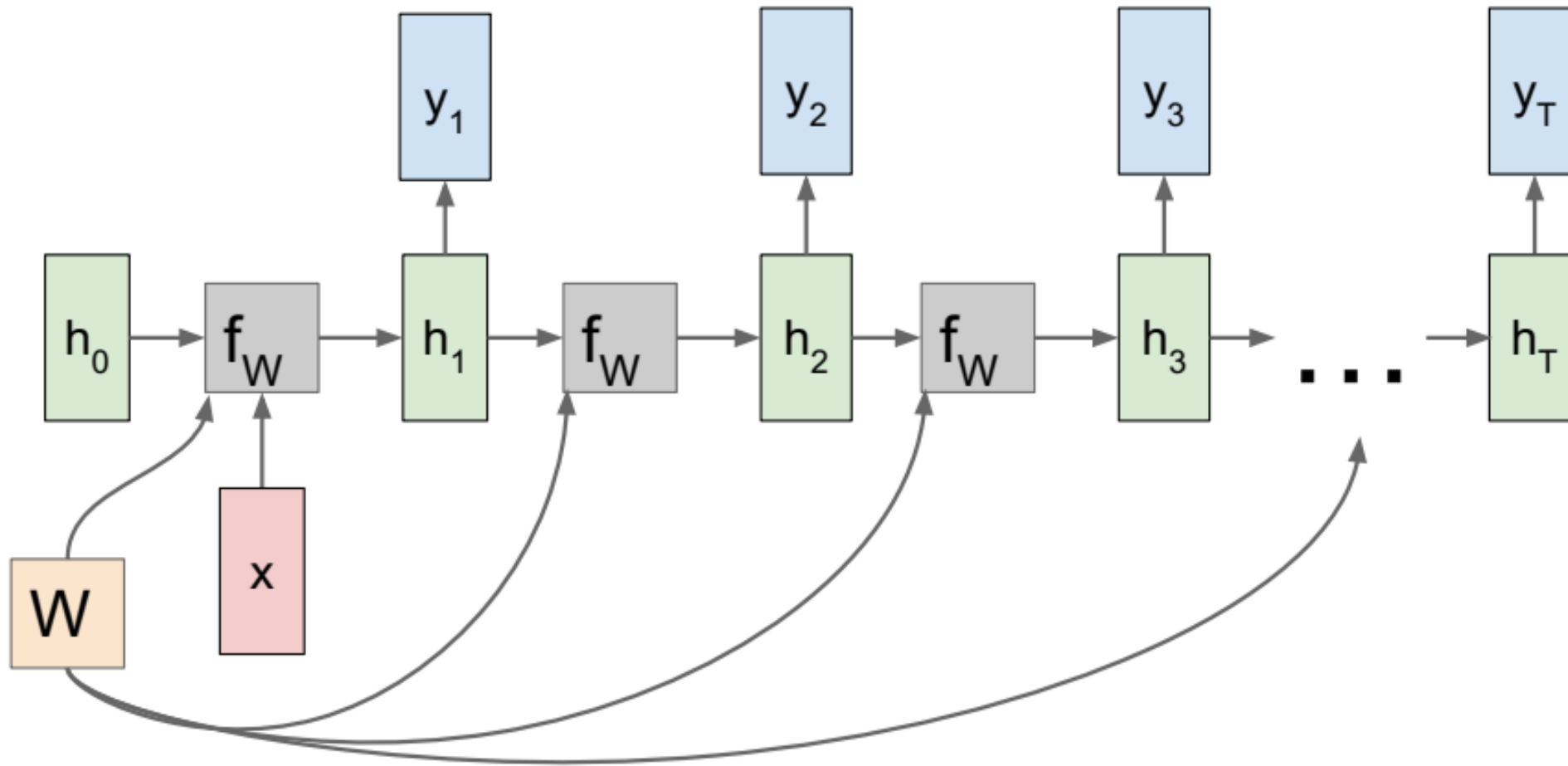
مدرس: محمدرضا محمدی

بهار ۱۴۰۲

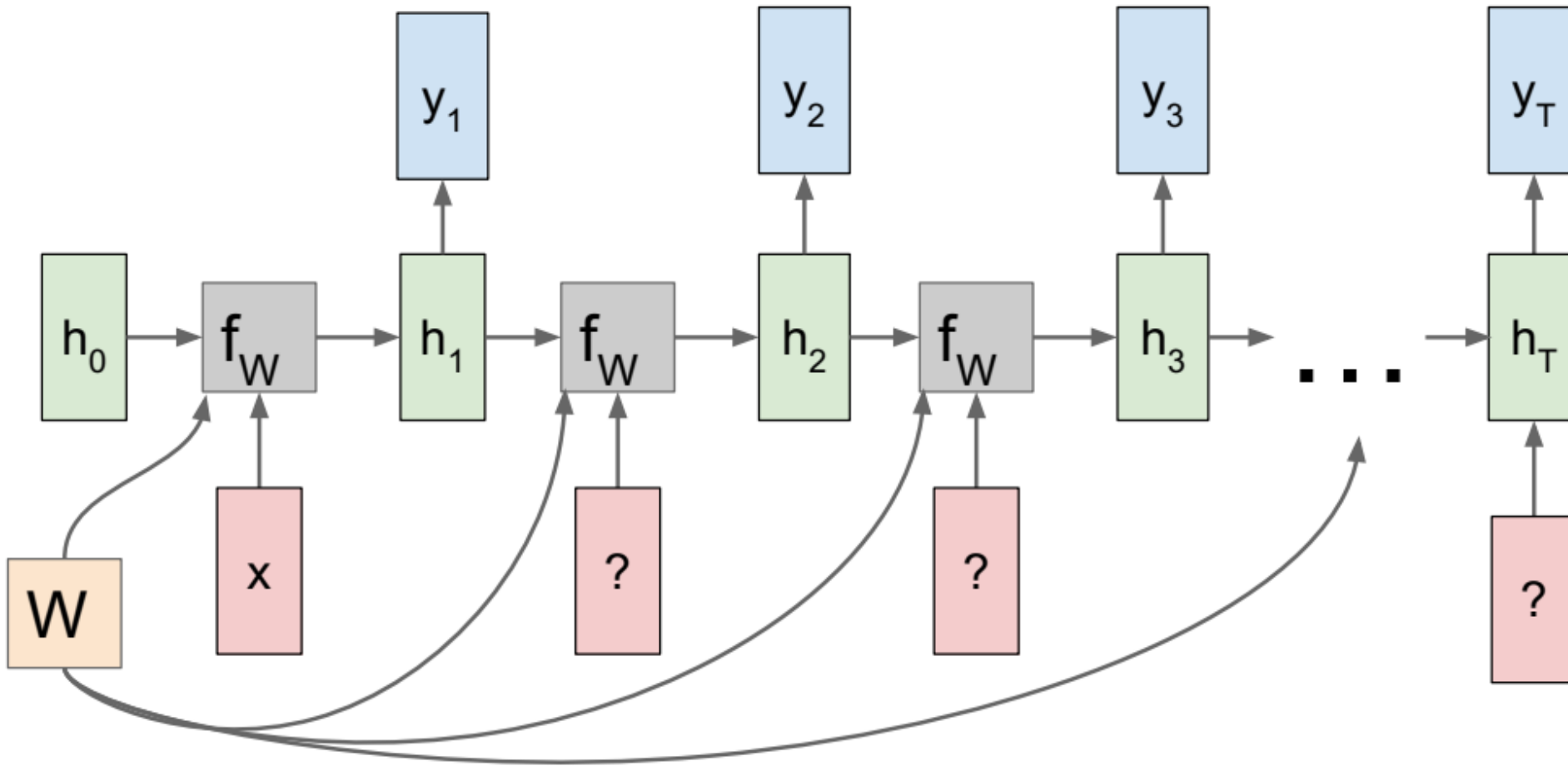
شبکه‌های عصبی بازگشتی

Recurrent Neural Networks

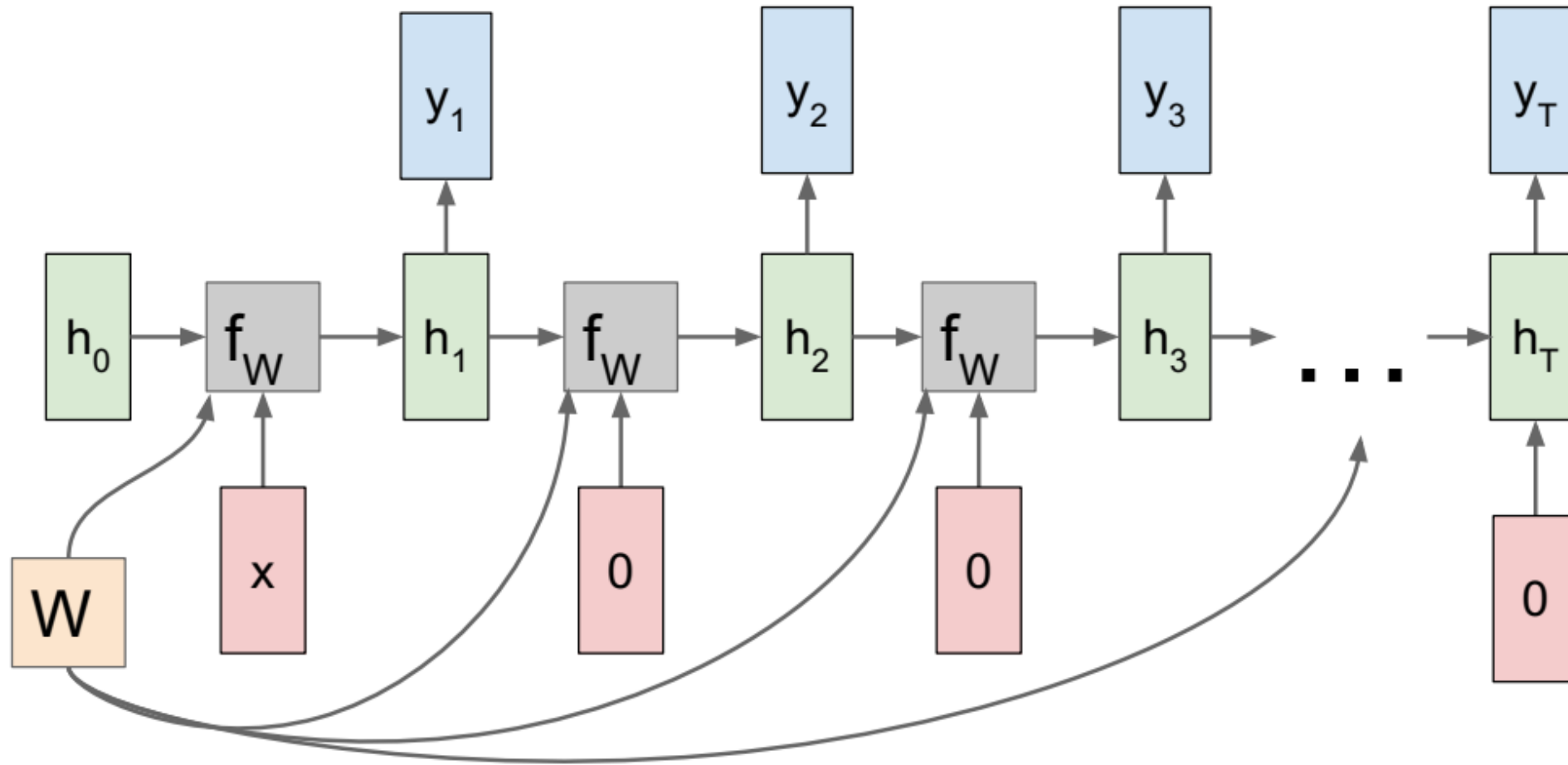
One to Many



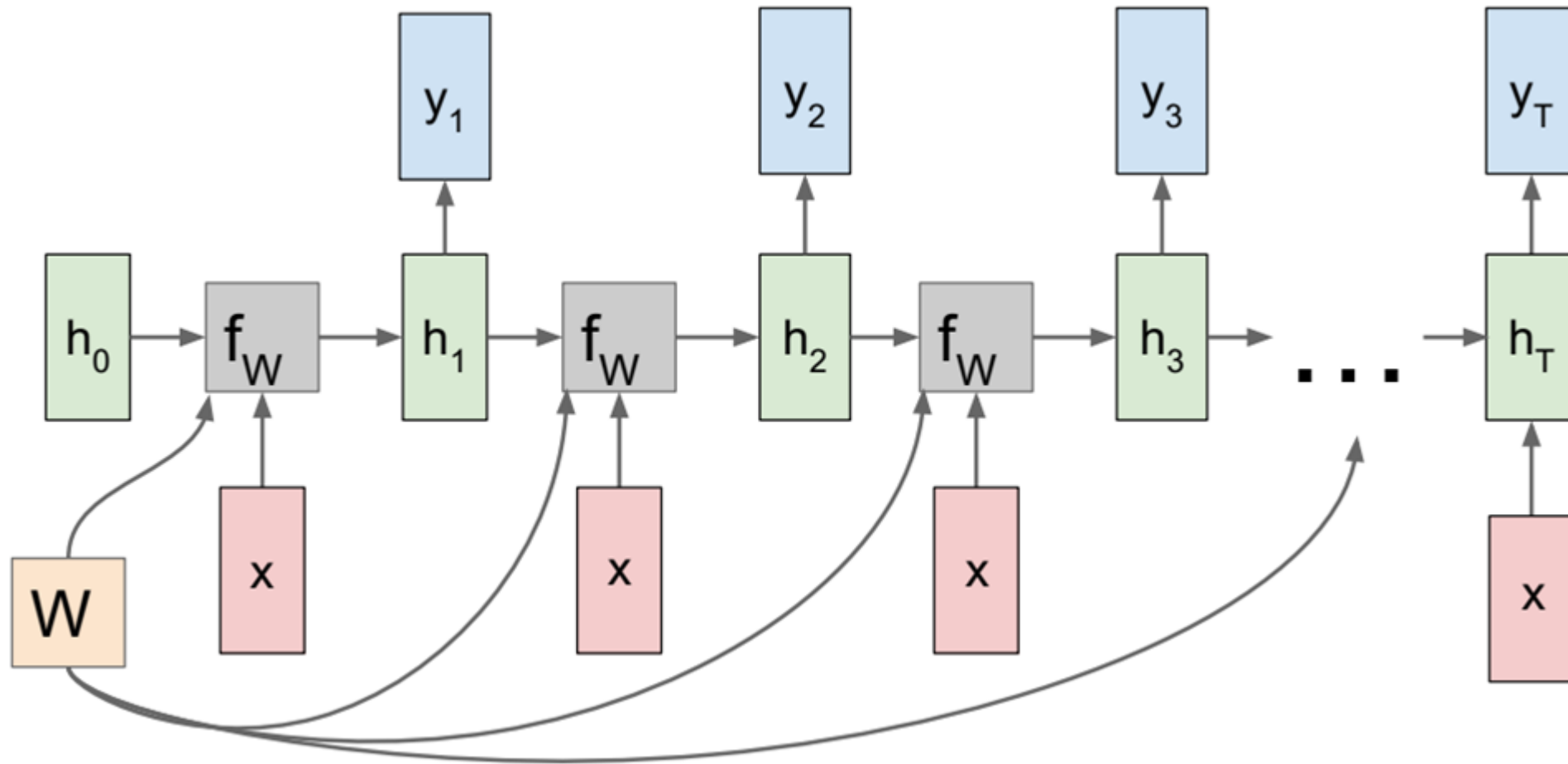
One to Many



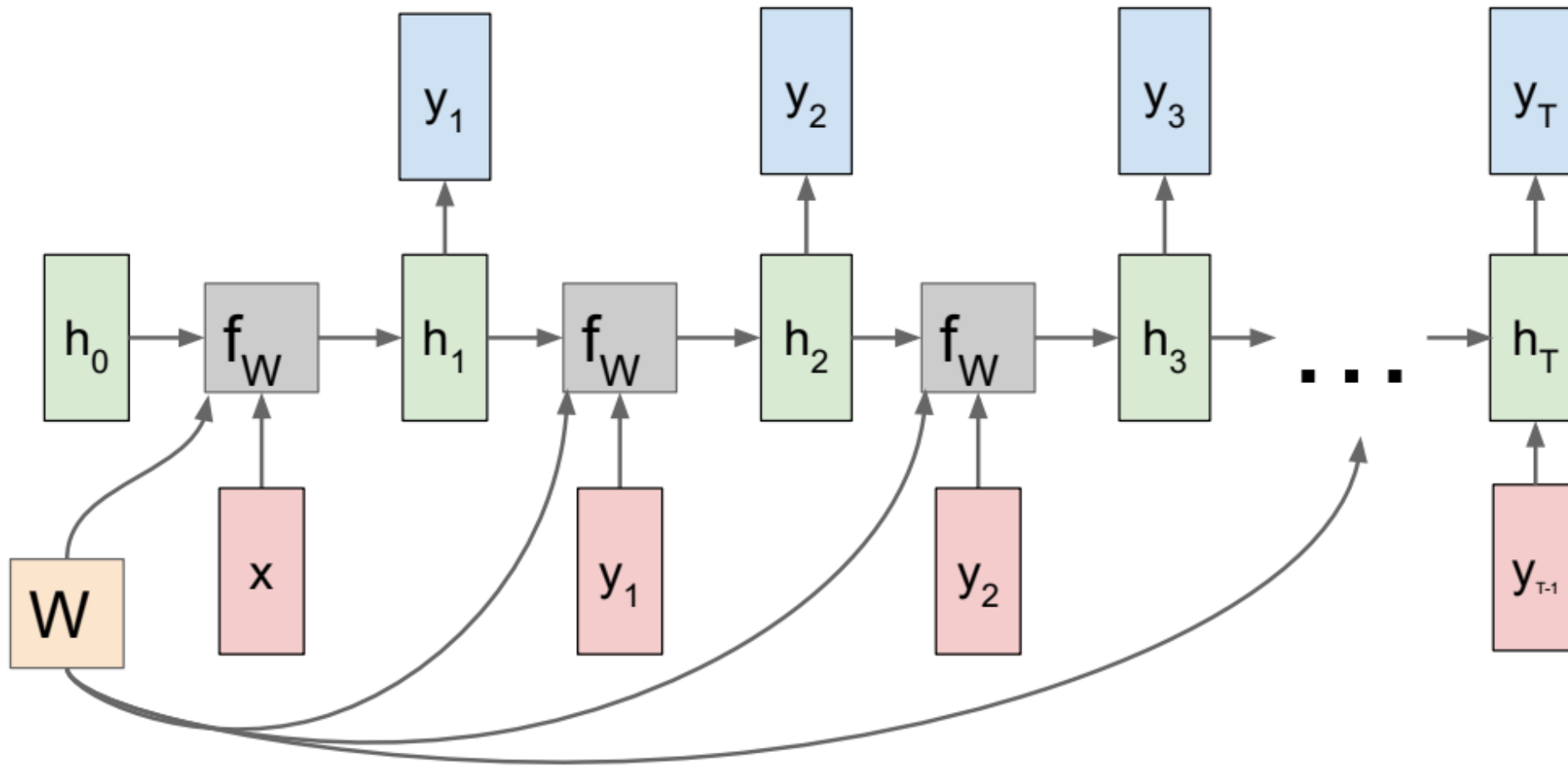
One to Many



One to Many

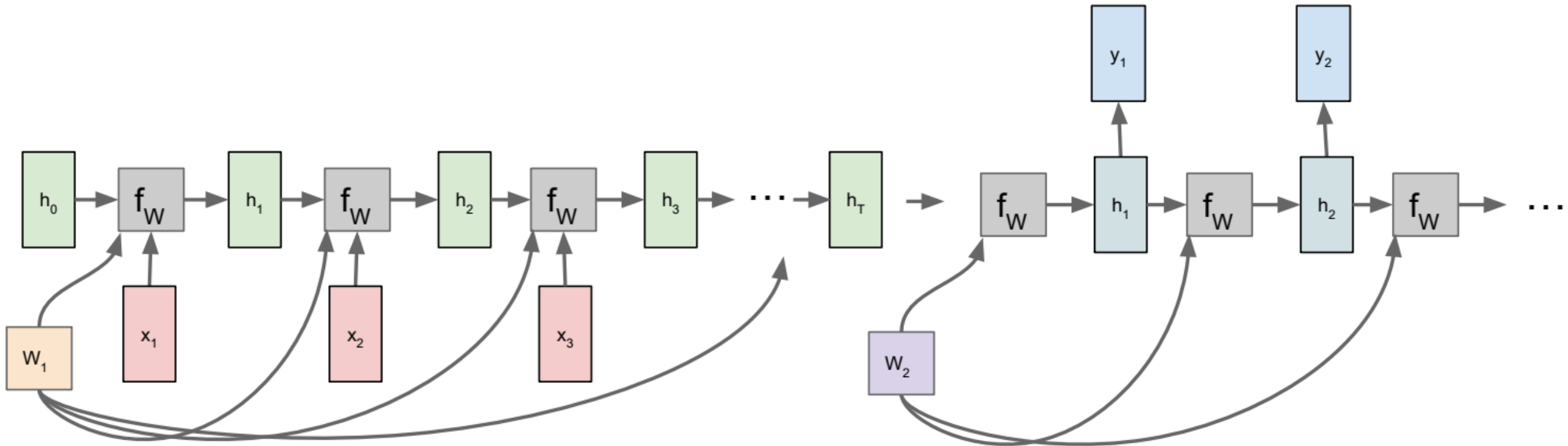


One to Many



Sequence to Sequence

- Many to One + One to Many
 - Many to one: Encode input sequence in a single vector
 - One to many: Produce output sequence from single input vector



محوشدگی و انفجار گرادیان

- در شبکه‌های بازگشتی برای دنباله‌های طولانی این مشکل بسیار جدی است
- می‌توان با برش گرادیان از انفجار گرادیان جلوگیری کرد

→ Forward Propagation →

$$I/p \xrightarrow{w_1} (b_1) \xrightarrow{w_2} (b_2) \xrightarrow{w_3} (b_3) \xrightarrow{w_4} (b_4) \xrightarrow{z_4} J$$

$$w_1 a_0 + b_1 = z_1 \quad a_1 = \text{sigmoid}(z_1)$$

$$w_2 z_1 + b_2 = z_2 \quad a_2 = \text{sigmoid}(a_1 w_2 + b_2)$$

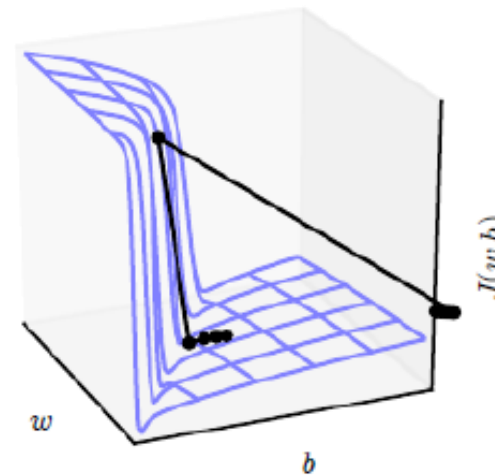
$$w_3 z_2 + b_3 = z_3 \quad a_3 = \text{sigmoid}(a_2 w_3 + b_3)$$

$$w_4 z_3 + b_4 = z_4 \quad a_4 = \text{sigmoid}(a_3 w_4 + b_4)$$

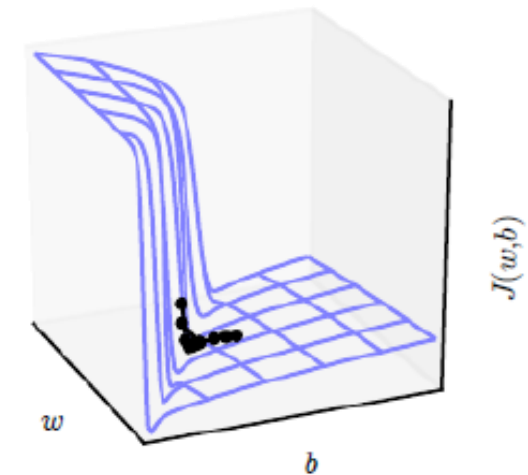
$$\frac{\partial J}{\partial b_1} = \frac{\partial J}{\partial a_4} \times \sigma'(z_4) w_4 \times \sigma'(z_3) w_3 \times \sigma'(z_2) w_2 \times \sigma'(z_1)$$

(Back Propagation Equation)

Without clipping



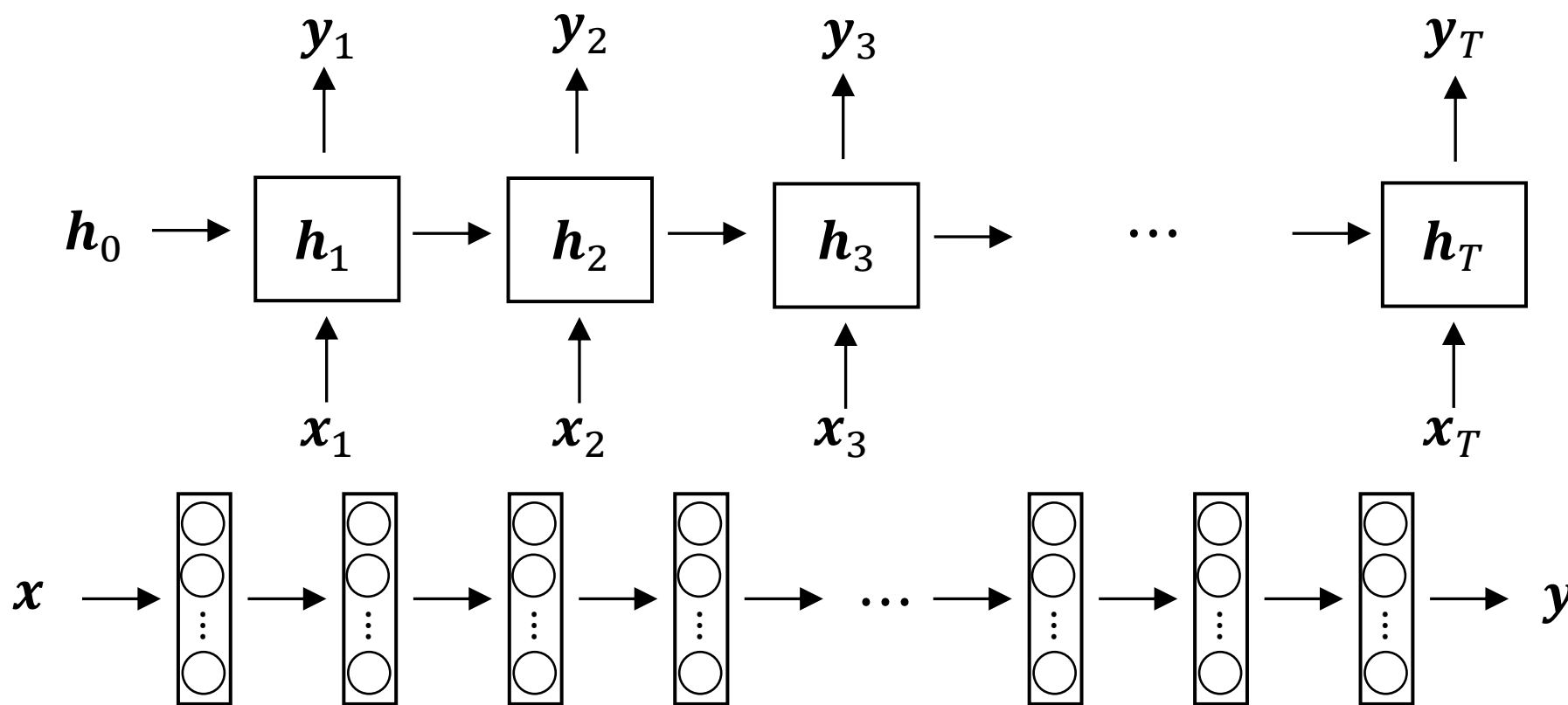
With clipping



محوشدگی گرادیان در RNNها

The **cat**, which already ate ..., **was** full.

The **cats**, which already ate ..., **were** full.

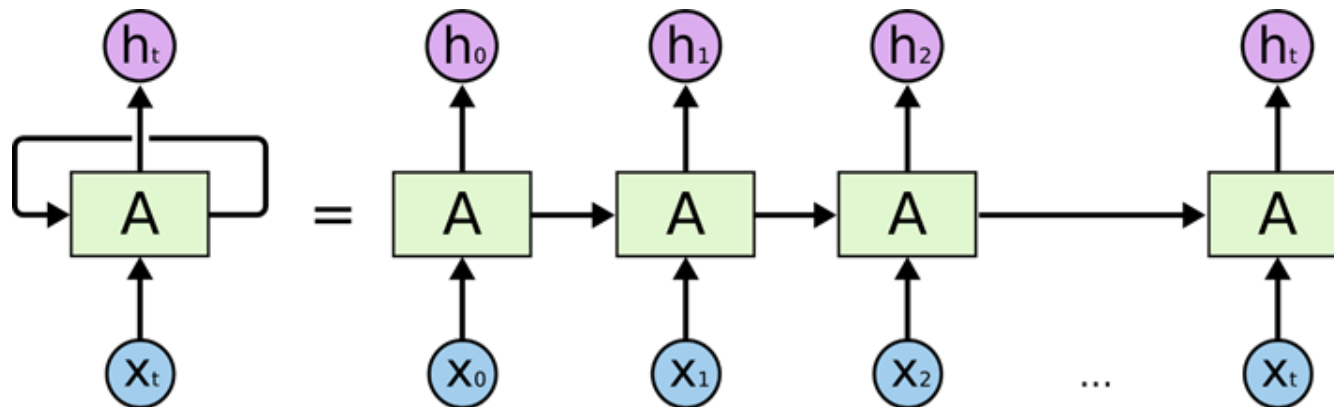


Gated RNNs

- شبکه‌های عصبی بازگشتی از حافظه کوتاه مدت رنج می‌برند
- RNN ممکن است اطلاعات مهم ابتدایی را نادیده بگیرد
- Gated RNNs مبتنی بر ایده ایجاد مسیرهایی در طول زمان هستند که از محوشدگی یا انفجار گرادیان جلوگیری می‌کنند

The **cat**, which already ate ..., **was** full.

The **cats**, which already ate ..., **were** full.



Customers Review 2,491

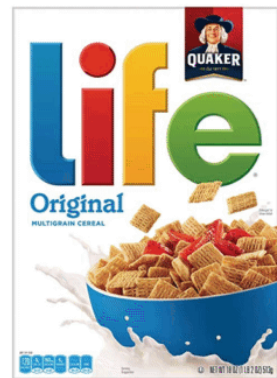


Thanos

September 2018

Verified Purchase

Amazing! This box of cereal gave me a perfectly balanced breakfast, as all things should be. I only ate half of it but will definitely be buying again!



A Box of Cereal
\$3.99

Gated Recurrent Units

GRU (simplified)

$$\tilde{\mathbf{h}}^{(t)} = \tanh(\mathbf{W}_{hh} \mathbf{h}^{(t-1)} + \mathbf{W}_{xh} \mathbf{x}^{(t)} + \mathbf{b}_h)$$

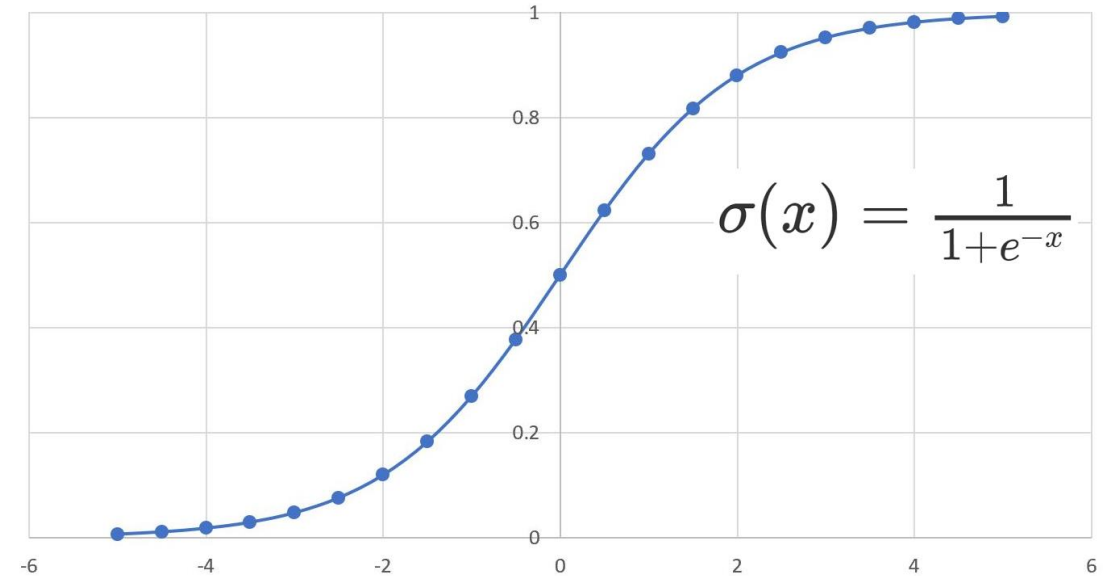
$$\mathbf{h}^{(t)} = \mathbf{u}^{(t)} \cdot \tilde{\mathbf{h}}^{(t)} + (1 - \mathbf{u}^{(t)}) \cdot \mathbf{h}^{(t-1)}$$

$$\mathbf{u}^{(t)} = \sigma(\mathbf{W}_{hu} \mathbf{h}^{(t-1)} + \mathbf{W}_{xu} \mathbf{x}^{(t)} + \mathbf{b}_u)$$

The cat, which already ate ..., was full.

Simple RNN

$$\mathbf{h}^{(t)} = \tanh(\mathbf{W}_{hh} \mathbf{h}^{(t-1)} + \mathbf{W}_{xh} \mathbf{x}^{(t)} + \mathbf{b}_h)$$



Gated Recurrent Units

GRU (simplified)

$$\tilde{\mathbf{h}}^{(t)} = \tanh(\mathbf{W}_{hh} \mathbf{h}^{(t-1)} + \mathbf{W}_{xh} \mathbf{x}^{(t)} + \mathbf{b}_h)$$

$$\mathbf{h}^{(t)} = \mathbf{u}^{(t)} \cdot \tilde{\mathbf{h}}^{(t)} + (1 - \mathbf{u}^{(t)}) \cdot \mathbf{h}^{(t-1)}$$

$$\mathbf{u}^{(t)} = \sigma(\mathbf{W}_{hu} \mathbf{h}^{(t-1)} + \mathbf{W}_{xu} \mathbf{x}^{(t)} + \mathbf{b}_u)$$

GRU

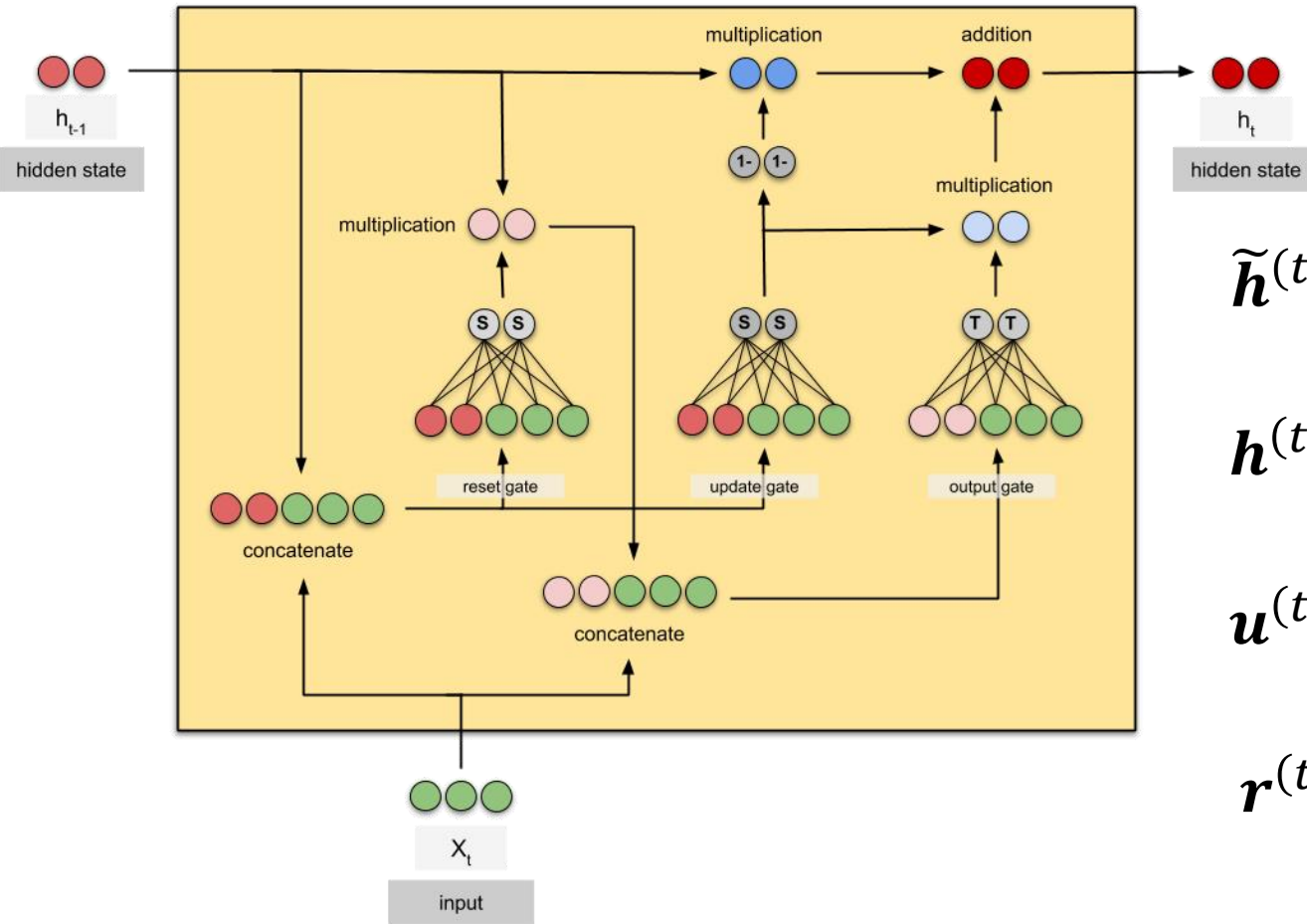
$$\tilde{\mathbf{h}}^{(t)} = \tanh(\mathbf{W}_{hh}(\mathbf{r}^{(t)} \cdot \mathbf{h}^{(t-1)}) + \mathbf{W}_{xh} \mathbf{x}^{(t)} + \mathbf{b}_h)$$

$$\mathbf{h}^{(t)} = \mathbf{u}^{(t)} \cdot \tilde{\mathbf{h}}^{(t)} + (1 - \mathbf{u}^{(t)}) \cdot \mathbf{h}^{(t-1)}$$

$$\mathbf{u}^{(t)} = \sigma(\mathbf{W}_{hu} \mathbf{h}^{(t-1)} + \mathbf{W}_{xu} \mathbf{x}^{(t)} + \mathbf{b}_u)$$

$$\mathbf{r}^{(t)} = \sigma(\mathbf{W}_{hr} \mathbf{h}^{(t-1)} + \mathbf{W}_{xr} \mathbf{x}^{(t)} + \mathbf{b}_r)$$

GRU



$$\tilde{h}^{(t)} = \tanh(W_{hh}(r^{(t)} \cdot h^{(t-1)}) + W_{xh} x^{(t)} + b_h)$$

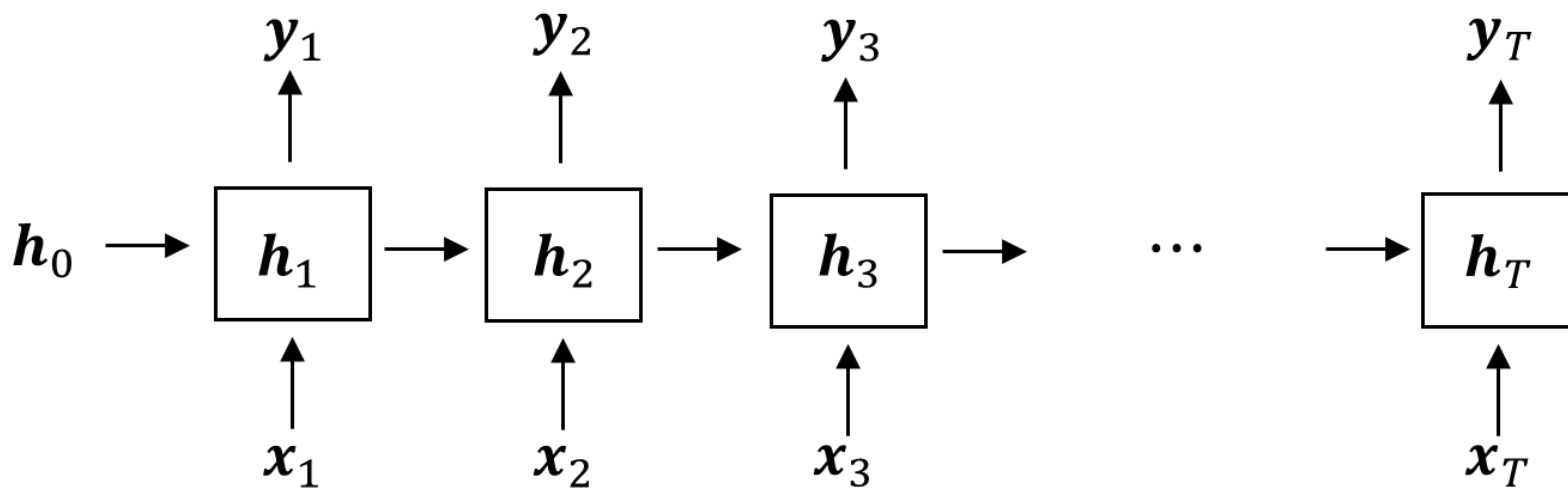
$$h^{(t)} = u^{(t)} \cdot \tilde{h}^{(t)} + (1 - u^{(t)}) \cdot h^{(t-1)}$$

$$u^{(t)} = \sigma(W_{hu} h^{(t-1)} + W_{xu} x^{(t)} + b_u)$$

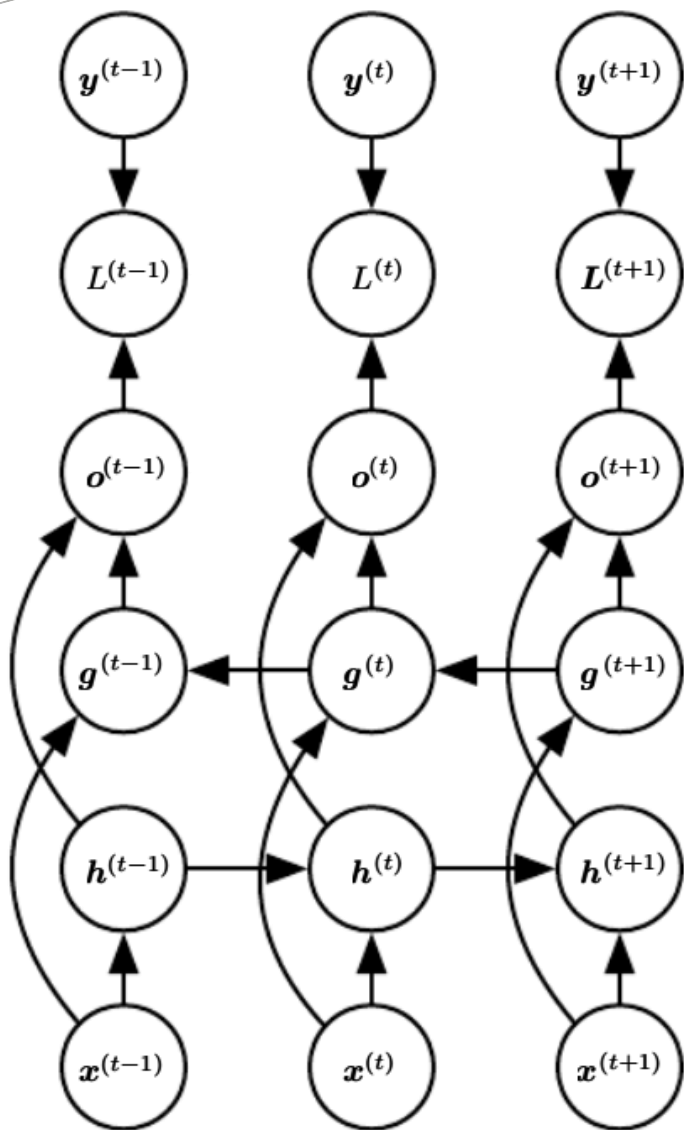
$$r^{(t)} = \sigma(W_{hr} h^{(t-1)} + W_{xr} x^{(t)} + b_r)$$

شبکه‌های بازگشتی دوجهته

- تمام RNN‌هایی که تا به حال بررسی کرده‌ایم دارای ساختار «علّی» هستند
 - حالت در زمان t فقط به ورودی‌های گذشته، $x(1), \dots, x(t-1)$ و ورودی فعلی $x(t)$ وابسته است
- در برخی از کاربردها، می‌توانیم از تمام مقادیر قبلی و بعدی برای پیش‌بینی $y(t)$ استفاده کنیم



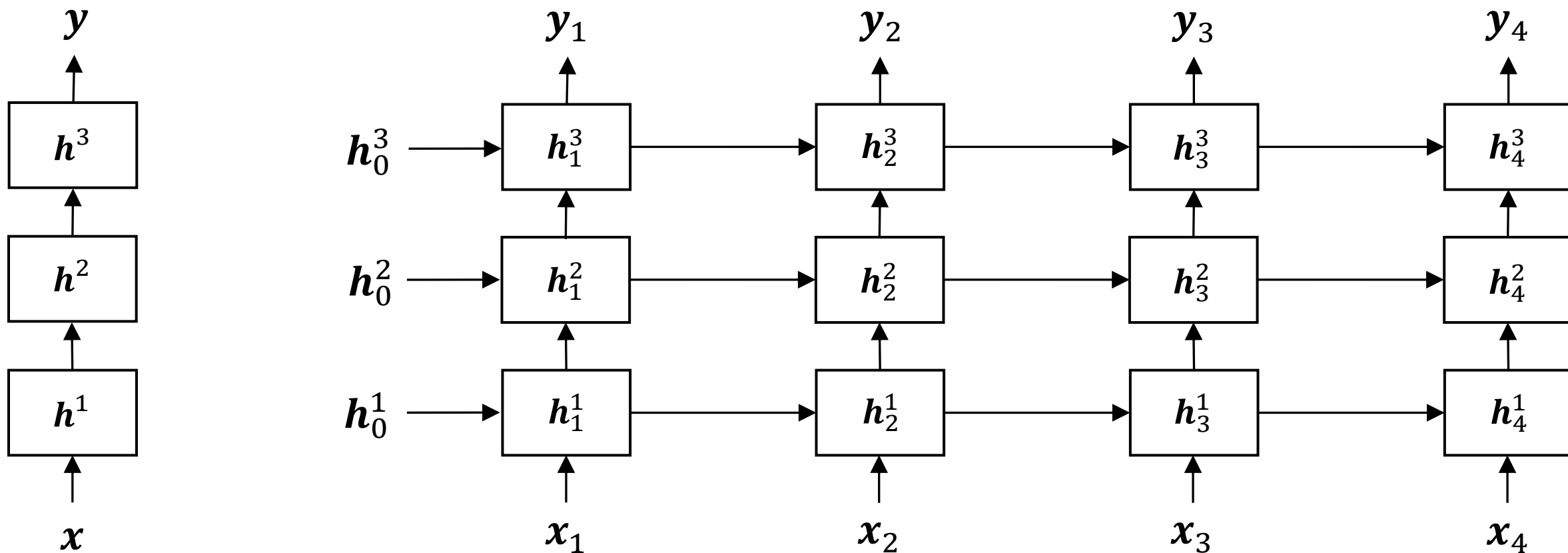
شبکه‌های بازگشتی دوجبهته



- RNNهای دو طرفه متداول شامل یک RNN رو به جلو از ابتدای دنباله و یک RNN رو به عقب از انتهای دنباله هستند
 - $h(t)$ حالت بخشی است که در زمان به جلو حرکت می‌کند
 - $g(t)$ حالت بخشی است که در زمان به عقب حرکت می‌کند
 - $o(t)$ هم به زمان گذشته و هم به زمان آینده بستگی دارد
- بیشترین حساسیت آن به زمان‌های نزدیک به t است
- هر کدام از SimpleRNN و LSTM و GRU در این ساختار دوجبهته قابل استفاده هستند

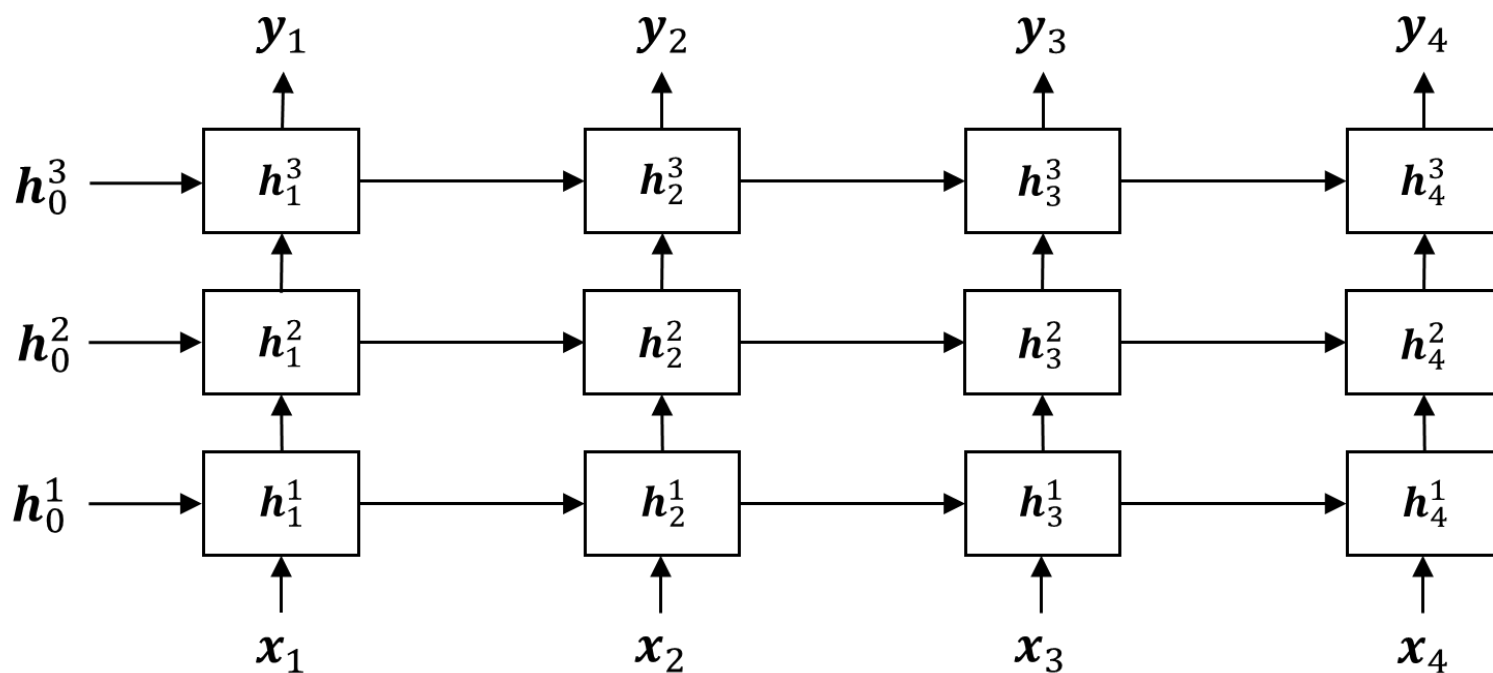
شبکه‌های بازگشتی عمیق

$$h_3^2 = \tanh(W^2 [h_2^2 \quad h_3^1] + b^2)$$



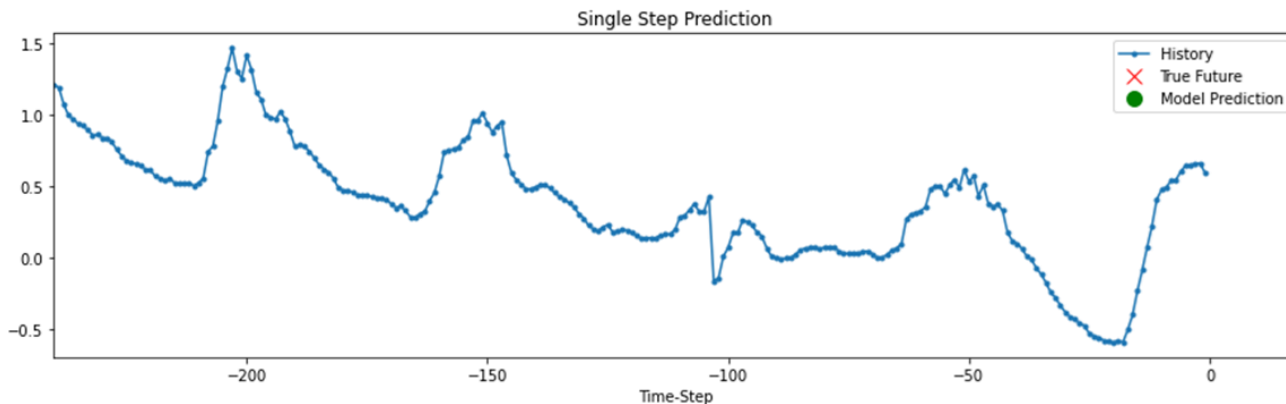
شبکه‌های بازگشتی عمیق

- برای RNN ها، داشتن سه لایه بسیار زیاد است
- بلوک‌های مورد استفاده می‌توانند SimpleRNN، GRU یا LSTM باشد
- هر کدام می‌توانند دوجهته باشند

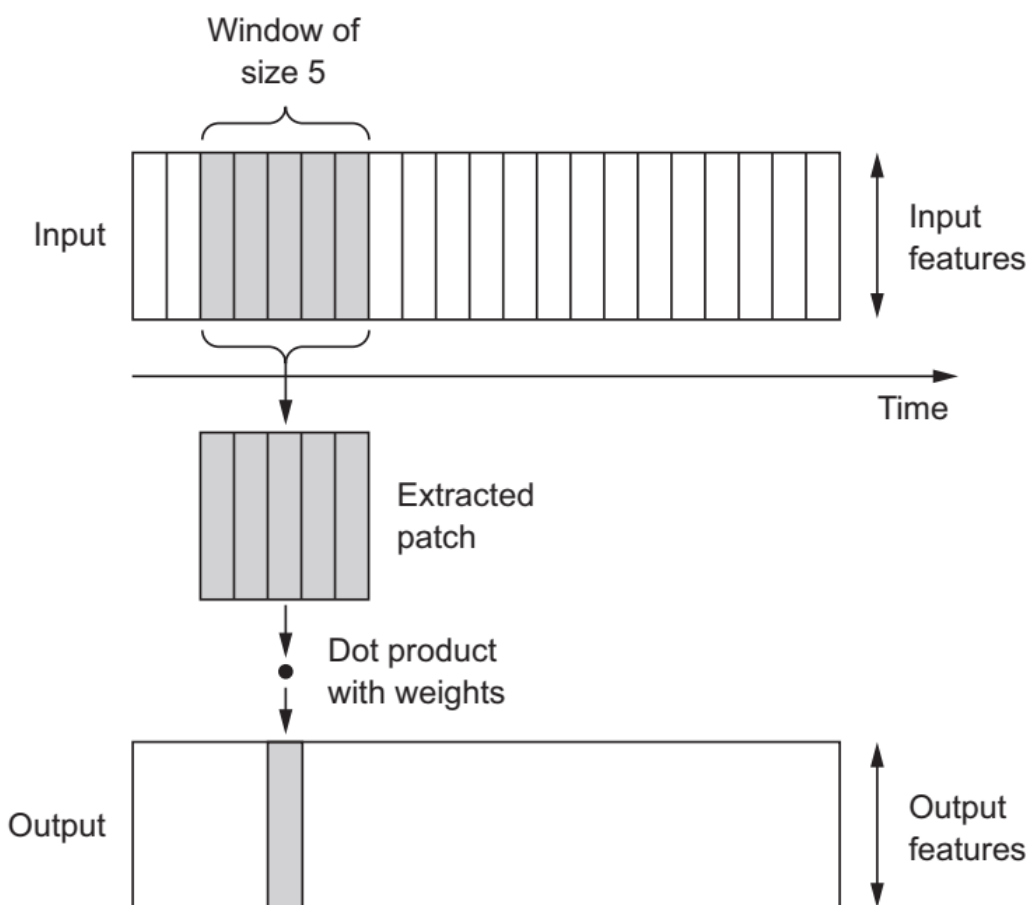


پردازش دنباله‌ها با لایه‌های کانولوشنی

- به همان دلایلی که لایه‌های کانولوشنی در بینایی ماشین بسیار موفق بوده‌اند، برای پردازش دنباله‌ها نیز موثر هستند
- زمان در دنباله‌ها مشابه با یک بعد مکانی در یک تصویر دو بعدی است
- شبکه‌های کانولوشنی یک‌بعدی می‌توانند با RNNها در مسائل پردازش دنباله‌ها رقابت کنند
 - معمولاً با هزینه محاسباتی بسیار کمتر



کانولوشن یک بعدی



- لایه های کانولوشنی یک بعدی می توانند الگوهای محلی را در یک دنباله تشخیص دهند
- الگویی که در یک موقعیت خاص در یک دنباله آموخته می شود، بعداً می تواند در موقعیت دیگری تشخیص داده شود، که باعث می شود لایه های کانولوشنی یک بعدی نسبت به جابجایی تغییرناپذیر باشند
- می توان از گام (Stride) و تجمیع (Pooling) هم استفاده کرد

ترکیب RNN ها و CNN ها

- از آنجایی که شبکه‌های کانولوشنی یک‌بعدی بخش‌های ورودی را به‌طور مستقل پردازش می‌کنند، بر خلاف RNN ها، به ترتیب مراحل زمانی (فراتر از مقیاس محلی مربوط به ابعاد پنجره) حساس نیستند

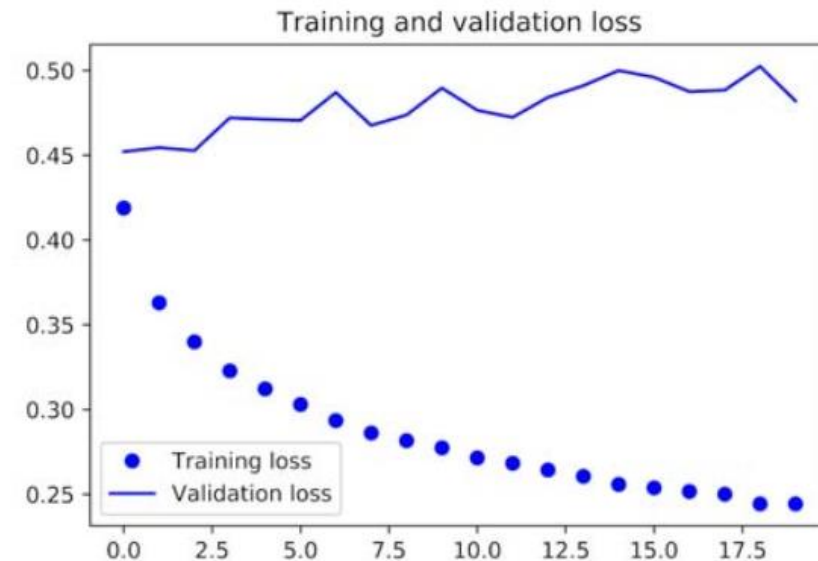
Listing 6.47 Training and evaluating a simple 1D convnet on the Jena data

```
from keras.models import Sequential
from keras import layers
from keras.optimizers import RMSprop

model = Sequential()
model.add(layers.Conv1D(32, 5, activation='relu',
                        input_shape=(None, float_data.shape[-1])))
model.add(layers.MaxPooling1D(3))
model.add(layers.Conv1D(32, 5, activation='relu'))
model.add(layers.MaxPooling1D(3))
model.add(layers.Conv1D(32, 5, activation='relu'))
model.add(layers.GlobalMaxPooling1D())
model.add(layers.Dense(1))

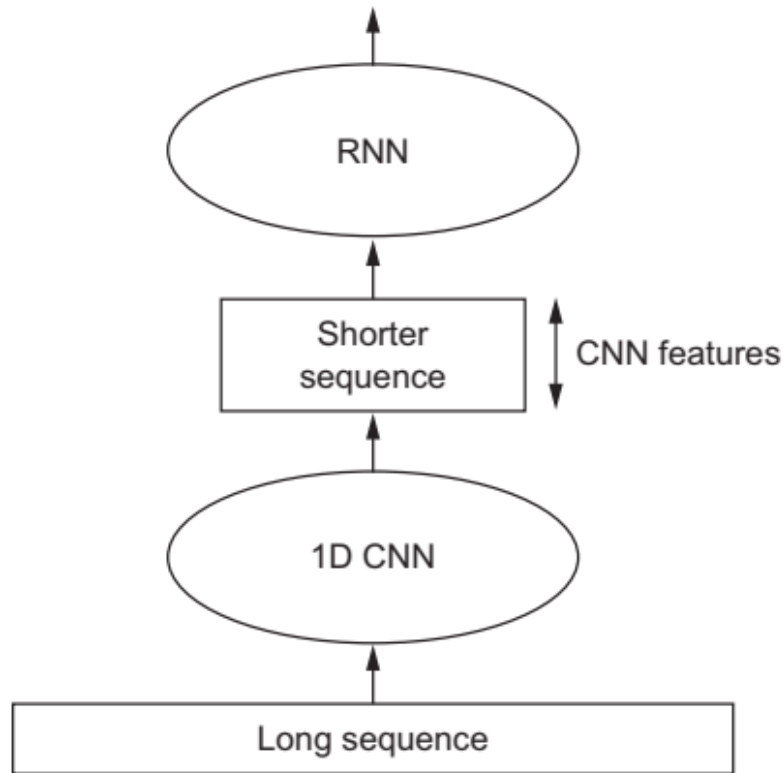
model.compile(optimizer=RMSprop(), loss='mae')
history = model.fit_generator(train_gen,
                             steps_per_epoch=500,
                             epochs=20,
                             validation_data=val_gen,
                             validation_steps=val_steps)
```

- پیش‌بینی دما توسط 1D CNN



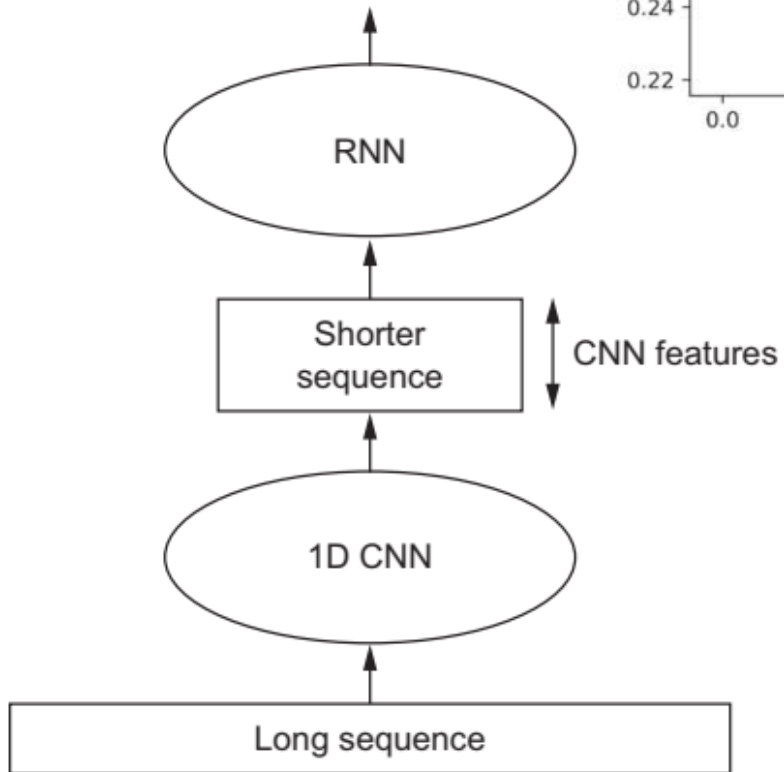
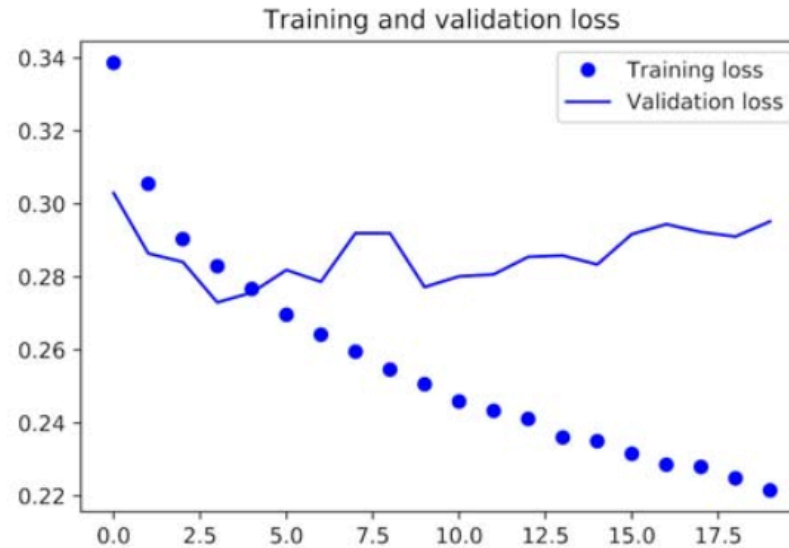
ترکیب RNN ها و CNN ها

- یک استراتژی برای ترکیب مزایای CNN ها (از جمله سرعت) با مزایای RNN ها (از جمله حساسیت به ترتیب مقادیر)، استفاده از CNN یک بعدی به عنوان مرحله پیش پردازش قبل از RNN است



- به خصوص برای پردازش دنباله های طولانی مفید است که نمی توان آنها را به طور موثری تنها با استفاده از RNN پردازش کرد

ترکیب RNN ها و CNN ها



Listing 6.49 Model combining a 1D convolutional base and a GRU layer

```
from keras.models import Sequential
from keras import layers
from keras.optimizers import RMSprop

model = Sequential()
model.add(layers.Conv1D(32, 5, activation='relu',
                        input_shape=(None, float_data.shape[-1])))
model.add(layers.MaxPooling1D(3))
model.add(layers.Conv1D(32, 5, activation='relu'))
model.add(layers.GRU(32, dropout=0.1, recurrent_dropout=0.5))
model.add(layers.Dense(1))

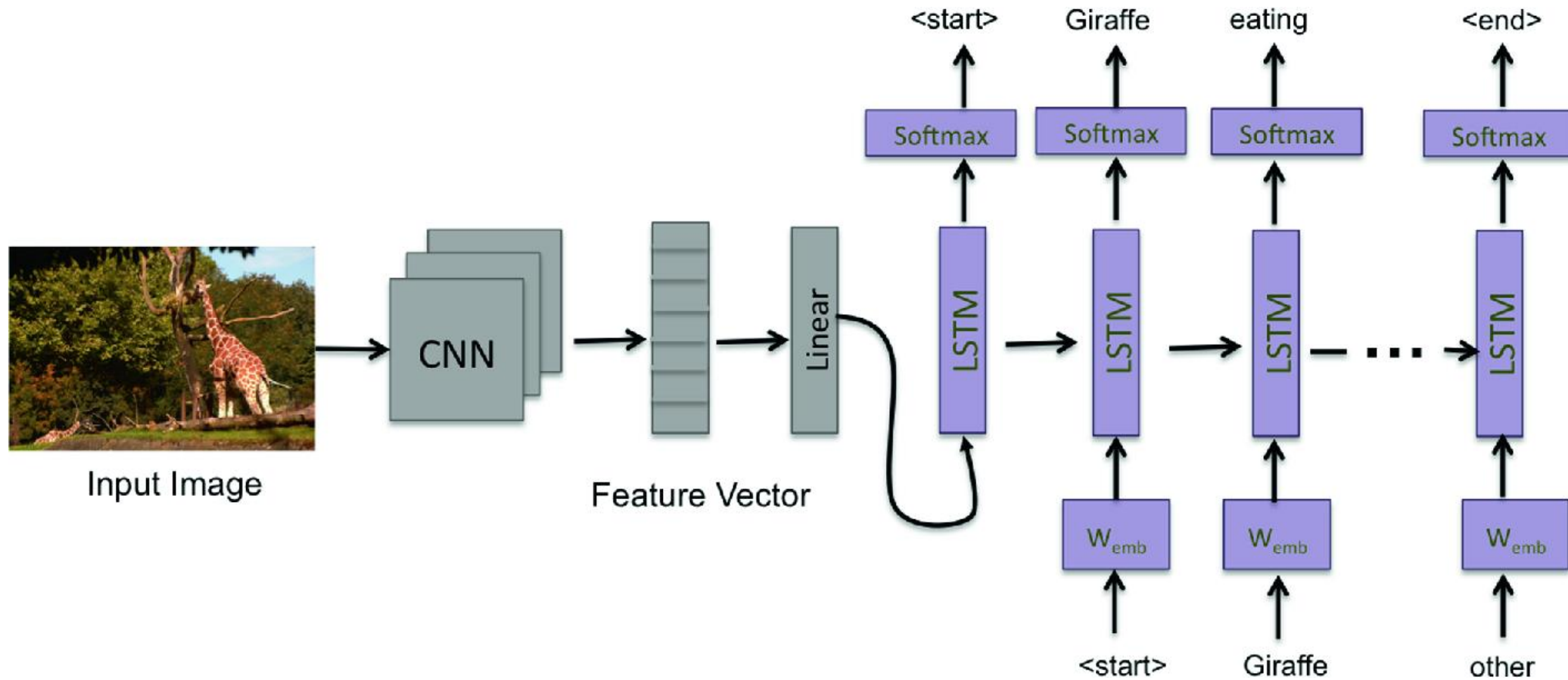
model.summary()

model.compile(optimizer=RMSprop(), loss='mae')
```


ترکیب RNN ها و CNN ها

- شبکه‌های کانولوشنی یک‌بعدی جایگزین سریع‌تری برای RNN ها در برخی مسائل هستند
- از آنجایی که RNN ها برای پردازش دنباله‌های بسیار طولانی، بسیار گران هستند، اما CNN های یک‌بعدی ارزان هستند، استفاده از یک CNN یک‌بعدی به عنوان گام پیش‌پردازش قبل از RNN برای کوتاه کردن دنباله و استخراج بازنمایی‌های موثر می‌تواند مفید باشد

شرح تصویر با CNN ها و RNN ها



تحلیل ویدئو با CNN ها و RNN ها

- ترکیبی از CNN و RNN پرکاربردترین معماری یادگیری عمیق برای لبخوانی خودکار (ALR) است

